

23BCS13733

Q1. Explain the role of interfaces and enums in software design with proper examples.

ANS: Interfaces: These are the elements of code which helps to enforce rules on classes.

These are used with classes so that a class must implement certain methods in itself but these don't define that how these methods behave.

Helps in:

- Solving Diamond problem
- Abstraction
- Loose coupling
- Better maintainability

Example:

```
interface Payment {  
    void pay(double amount);  
}  
  
class CreditCardPayment implements Payment {  
    public void pay(double amount) {  
        System.out.println("Paid " + amount + " using Credit Card");  
    }  
}  
  
class UpiPayment implements Payment {  
    public void pay(double amount) {  
        System.out.println("Paid " + amount + " using UPI");  
    }  
}
```

Enums: It is a special type of data type that represents fixed set of constants. Like when choosing a 'state' in form then we predefine all the states name to avoid error and typos.

Helps in:

- Type safety
- Improved Readability
- Centralized Control
- Better Maintainability

Example:

```
enum OrderStatus {
    PLACED,
    SHIPPED,
    DELIVERED,
    CANCELLED
}

OrderStatus status = OrderStatus.SHIPPED;

if (status == OrderStatus.DELIVERED) {
    System.out.println("Order completed");
}
```

Q2. Discuss how interfaces enable loose coupling with example.

ANS: Loose coupling: Loose coupling means that classes depend on abstractions rather than concrete implementations. Changes in one class do not force changes in other classes.

Without an interface, a class often depends directly on a specific implementation (Concrete Class A depends on Concrete Class B). This is tight coupling. If you want to swap Class B for Class C, you have to rewrite parts of Class A.

By introducing an interface, Class A now depends on an abstraction. It doesn't care who fulfills the contract, as long as the methods are there.

Key Benefits

- **Swapability:** You can change the underlying logic (e.g., switching from a MySQL database to a MongoDB) without touching the high-level business logic.
- **Parallel Development:** One developer can write the code that *uses* the interface while another writes the *implementation*.
- **Testability:** You can swap a real, heavy service (like a credit card processor) with a "Mock" or "Fake" version for testing purposes.

Example:

```
public interface PaymentProcessor {
    void processPayment(double amount);
}

public class PayPalImpl implements PaymentProcessor {
    public void processPayment(double amount) { /* PayPal logic */ }
}
```

```
public class StripeImpl implements PaymentProcessor {
    public void processPayment(double amount) { /* Stripe logic */ }
}

public class OrderProcessor {
    private PaymentProcessor payment;

    public OrderProcessor(PaymentProcessor payment) {
        this.payment = payment;
    }

    public void checkout(double total) {
        payment.processPayment(total);
    }
}
```